



TORNADOVM

MANCHESTER
1824

The University of Manchester

TornadoVM

The Journey from Academia to Open-Source Success

Thanos Stratikopoulos
Beehive Lab – The University of Manchester

OC for Research – 21 April 2026

tornadovm.org • github.com/bee-hive-lab/TornadoVM • [@tornadovm](https://twitter.com/tornadovm)

About me

MANCHESTER
1824

The University of Manchester



Thanos Stratikopoulos

Research Fellow

The University of Manchester

✉ athanasios.stratikopoulos@manchester.ac.uk

X [@thanos_str](https://twitter.com/thanos_str)

in <https://www.linkedin.com/in/stratika/>

🦋 [@stratika.bsky.social](https://bsky.app/profile/stratika.bsky.social)

🐙 [@stratika](https://github.com/stratika)

Agenda

1 Origins & Evolution

From PhD research to production-ready framework

2 Research & Impact

The R&D cycle

3 Core Technology

TornadoVM, tooling and deployment

4 Use cases & EU RIA Projects

Use cases, AERO, TANGO, P2CODE, ELEGANT, E2Data

5 Open-Source Practices

PRs, CoC, CLA, CI Workflows, Release Cycles

6



Origins & Evolution

How It All Started

A decade-long journey from research to production



PhD 2013

Manchester research project,
initially named Jacc



Open Source 2018

Transitioned to Graal,
open-sourced on GitHub



Production 2025

ESA GAIA deployment,
30+ contributors



Originally based on SOOT | Evolved through EU & UKRI funded projects | Now v4.0 with 30+ releases |
1.4k+ GitHub stars | 10,000+ commits



Research & Impact

The R&D Cycle

Top research features and key academic contributions underpinning TornadoVM

1. Hypothesis

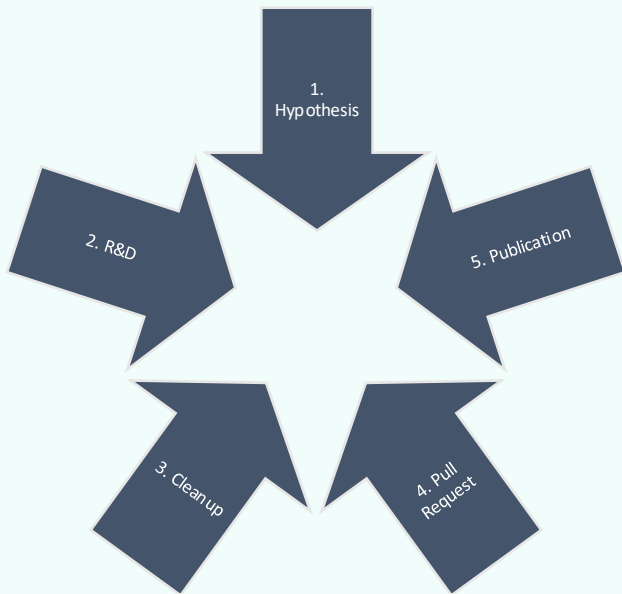
- Identify challenges
- Define a testable research question

2. R&D

- Prototype Implementation
- Early design exploration
- First results & validation

3. Code Cleanup

- Refactor research prototype
- Improve structure & maintainability
- Add tests & documentation



4. Pull Request

- GitHub PR to TornadoVM
- CI on all backends & JDKs
- Code review & merge

5. Publication

- Writing paper
- Final Evaluation & Results
- Peer review

Research and open source form a continuous feedback loop — papers validate ideas, production code sustains impact.

Research Publications

Top research features and key academic contributions underpinning TornadoVM

20+

Peer-reviewed
Publications

7+

Top-tier Venues

VEE • MPLR • FCCM • VLDB • VMIL •
Programming • MoreVMs • Springer

Unified Shared Memory: Friend or Foe? Understanding the Implications of Unified Memory on Managed Heaps

Juan Fumero
juan.fumero@manchester.ac.uk
The University of Manchester
United Kingdom

Florin Blanu
florin.blanu@axelera.ai
Axelera AI
Netherlands

Athanasios Stratikopoulos
(first) [last]@manchester.ac.uk
The University of Manchester
United Kingdom

Abstract

Adapting heterogeneous execution on GPUs and PGAs in managed runtime systems, such as Java, is a challenging task due to the complexities of the underlying virtual machine. The majority of the current work has been focusing on compiler toolchains to solve the challenge of transparent just-in-time compilation of different code segments onto the accelerators. However, apart from providing automatic code generation, another paramount challenge is the seamless interoperability between the host memory manager and the Garbage Collector (GC). Currently, heterogeneous programming models that run on top of managed runtime systems, such as Aparapi and TornadoVM, need to block the GC when running native code (e.g., JNI code) in order to prevent the GC from moving data while the native code is still running on the hardware accelerator.

To tackle the difficulty of locking the GC while the GPU operates, this paper proposes a novel Unified Memory (UM) memory allocator for heterogeneous programming frameworks for managed runtime systems. In this paper, we show how, by providing small changes to a Java runtime system, automatic memory management can be enhanced to perform object reclamation not only on the host, but also on the device. This is done by altering the Java Virtual Machine's object heap in unified memory which is visible to all hardware accelerators. In this manner, although explicit data migration between the host and the device is still required

^{*}Work done while he was associated at The University of Manchester.

to ensure data consistency: we enable transparent page migration of Java heap-allocated objects between the host and the accelerator, since our UM system is aware of pointers and object migration due to GC collections. This technique has been implemented in the context of MatioVM, an open source research VM for Java written in Java. We evaluated our approach on a discrete and an integrated GPU, showcasing under which conditions UM can benefit execution across different benchmarks and configurations. We concluded that when hardware acceleration is not employed, UM does not pose significant overheads unless memory intensive workloads are encountered which can exhibit up to 12% (worst case) and 2% (average) slowdowns. In addition, if hardware acceleration is used, UM can achieve up to 9.3x speedup compared to the non-UM baseline implementation for integrated GPUs.

CCS Conception: Computing methodologies → Parallel programming languages • Software and its engineering → Just-in-time compilers, Runtime environments.

Keywords: CUDA, Data Transfers, GPUs, JVM, LevelZero, Memory Management, Unified Memory, Virtual Machines

ACM Reference Format:

Juan Fumero, Florin Blanu, Athanasios Stratikopoulos, Steve Dohrmann, Sandhya Viswanathan, and Christos Kotselidis. 2023. Unified Shared Memory: Friend or Foe? Understanding the Implications of Unified Memory on Managed Heaps. In *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming*.

Enabling Transparent Acceleration of Big Data Frameworks Using Heterogeneous Hardware

Maria Xekalaki[‡], Juan Fumero[‡], Athanasios Stratikopoulos[‡], Katerina Doka[‡], Christos Katsakioris[‡], Constantinos Bitsakos[‡], Nectarios Koziris[‡], Christos Kotselidis[‡]
‡The University of Manchester, UK
(first) [last]@manchester.ac.uk
[‡]National Technical University of Athens, Greece
{doka, ktsak, kbtsak, nkoziris}@elab.ece.ntua.gr

ABSTRACT

The ever-increasing demand for high performance Big Data analytics and data processing, has paved the way for heterogeneous hardware accelerators, such as Graphics Processing Units (GPUs) and Field Programmable Gate Arrays (FPGAs), to be integrated into modern Big Data platforms. Currently, this integration comes at the cost of programmability since the end-user Application Programming Interface (APIs) must be altered to access the underlying heterogeneous hardware. For example, current Big Data frame-

The source code, data, and/or other artifacts have been made available at <https://github.com/maisonni/Flink-TornadoVM-Artifact>.

1 INTRODUCTION

The staggering increase in the generation rate of data - which is predicted to reach 161 terabytes by 2025 [59] - has posed new challenges and opportunities regarding high-performance and energy efficient data analytics. To address these challenges, heterogeneous hardware acceleration such as Graphics Processing Units (GPUs)

Scaling Up Performance of Managed Applications on NUMA Systems

Orion Papadakis[†], Andreas Andronikakis[‡], Nikos Foutiris[‡], Michail Papadimitriou[†], Athanasios Stratikopoulos[‡], Foivos Zakkak[‡], Polychronis Xekalakakis[‡], Christos Kotselidis[‡]
[†]The University of Manchester, [‡]Red Hat, [‡]NVIDIA
first.last@manchester.ac.uk, frakkak@redhat.com, pxekalakakis@nvidia.com

Abstract

Scaling up the performance of managed applications on Non-Uniform Memory Access (NUMA) architectures has been a challenging task, as it requires a good understanding of the underlying architecture and managed runtime environments (MRE). Prior work has studied this problem from the scope of specific components of the managed runtime, such as the Garbage Collectors, as a means to increase the NUMA awareness in MREs.

In this paper, we follow a different approach that complements prior work by studying the behavior of managed applications on NUMA architectures during mutation time.

[†]As first, we reformulate a shared-memory state that facilitates

oblivious of the NUMA characteristics. Therefore, two research questions that arise are: *Which are the limiting factors of MREs in the context of NUMA systems? Which parts of MREs should be enriched with NUMA awareness?*

In recent years, several studies have attempted to address the aforementioned questions in the context of the Java Virtual Machine (JVM) [2, 9, 10, 22]. These research studies have mainly focused on specific characteristics of the Garbage Collector (GC) towards enhancing its scalability. However, the performance of Java applications on NUMA architectures is not only a subject of GC scalability. To comprehend this, we have compared the NUMA performance (excluding GC time) of the Daquao [3] and Renaissance [24] benchmark

Cross-Language Interoperability of Heterogeneous Code

Athanasios Stratikopoulos
OxoML
The University of Manchester
Manchester, United Kingdom
(first) [last]@manchester.ac.uk

Florin Blanu^{*}
Seattle, United States
flblanu@oxotml.ai

Juan Fumero
The University of Manchester
Manchester, United Kingdom
juan.fumero@manchester.ac.uk

Maria Xekalaki
The University of Manchester
Manchester, United Kingdom
maria.xekalaki@manchester.ac.uk

Orion Papadakis
The University of Manchester
Manchester, United Kingdom
orion.papadakis@manchester.ac.uk

Christos Kotselidis
The University of Manchester
Manchester, United Kingdom
christos.kotselidis@manchester.ac.uk

ABSTRACT

In recent years, the Java Virtual Machine has evolved from a cross-SA virtualization layer to a system that can also offer multilingual support. GraalVM paved the way to enable the interoperability of Java with other programming languages, such as Java, Python, R and even C++, that can run on top of the Truffle framework in a unified manner. Additionally, there have been numerous academic and industrial endeavors to bridge the gap between the JVM and modern heterogeneous hardware resources. All these efforts beckon the opportunity to use the JVM as a unified system that enables interoperability between multiple programming languages and multi-heterogeneous hardware resources.

ACM Reference Format:

Athanasios Stratikopoulos, Florin Blanu, Juan Fumero, Maria Xekalaki, Orion Papadakis, and Christos Kotselidis. 2023. Cross-Language Interoperability of Heterogeneous Code. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym '23)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Previous research work has showcased how Java and particularly the Java Virtual Machine (JVM) can be used as a platform that facilitates the implementation of new programming languages [11]. The emergence of GraalVM along with its unified approach to reuse

Beehive SPIR-V Toolkit

A Composable and Functional API for Runtime SPIR-V Code Generation

Juan Fumero
juan.fumero@manchester.ac.uk
The University of Manchester
United Kingdom

György Rethy^{*}
gretthy@student.ethz.ch
ETH Zurich
Switzerland

Athanasios Stratikopoulos
(first) [last]@manchester.ac.uk
The University of Manchester
United Kingdom

Nikos Foutiris
nikos.foutiris@manchester.ac.uk
The University of Manchester
United Kingdom

Christos Kotselidis
christos.kotselidis@manchester.ac.uk
The University of Manchester
United Kingdom

Abstract

The Standard Portable Intermediate Representation (SPIR-V) is a low-level binary format designed for representing shaders and compute kernels that can be consumed by OpenGL for computing kernels, and Vulkan for graphics rendering. As a binary representation, SPIR-V is meant to be used by compilers and runtime systems, and is usually performed by C/C++ programs and the LLVM software and compiler

To demonstrate the use of our framework, we showcase the integration of the SPIR-V Beehive Toolkit in the context of the TornadoVM, a Java framework for automatically offloading and running Java programs on heterogeneous hardware. We show that, by using the SPIR-V Beehive Toolkit, TornadoVM is able to compile code 3x faster than its existing OpenGL C JIT compiler, and it performs up to 1.52x faster than the existing OpenGL C backend in TornadoVM.



3

What is TornadoVM?

A production-ready, open-source GPU acceleration framework for Java



TORNADOVM



@tornadovm



@tornadovm.org



<https://tornadovm.org>



<https://github.com/beehive-lab/TornadoVM>



SCAN ME

A JVM plugin that accelerates Java methods on heterogeneous hardware!

Features:

- Open source
- Platform agnostic API
- Automatic code optimization at runtime

A Mature Ecosystem

Balancing academic innovation, open-source development, and real-world impact



Multi-Backend

OpenCL, CUDA/PTX,
SPIR-V, Metal

CPUs, GPUs, FPGAs



JDK Compatibility

OpenJDK, GraalVM,
Temurin, Mandrel,
Zulu, SapMachine,
Amazon Corretto,
Microsoft OpenJDK



AI-Ready

GPULlama3.java,
LangChain4j, Quarkus,
Maven Central



Rich Tooling

TornadoInsight plugin,
TornadoPulse, TornadoViz,
Nsight, VTune, Docker



Deployed in ESA GAIA mission • Used across 7+ EU & UKRI funded research projects

JDK 21+ required. Distributed via SDKMAN! and tornadovm.org/downloads. Apache 2.0 License.

Quick Install TornadoVM

Three ways to get started

1 Through SDKMAN!

Recommended



```
$ sdk list tornadovm
$ sdk install tornadovm 4.0.0-openc1
```

2 Download the SDK directly

<https://www.tornadovm.org/downloads>

3 Clone & build from source

```
$ git clone https://github.com/beehive-lab/TornadoVM.git
$ cd TornadoVM
$ ./bin/tornadovm-installer
$ source setvars.sh
```

Available Versions

4.0.0-jdk25-met
4.0.0-jdk25-ope
4.0.0-jdk21-met
4.0.0-jdk21-ope

latest

Download TornadoVM SDK

4.0.0 (JDK 21)

linux/x86_64

Download

For Maven projects (pom.xml)

```
<dependencies>
  <dependency>
    <groupId>io.github.beehive-lab</groupId>
    <artifactId>tornado-api</artifactId>
    <version>4.0.0</version>
  </dependency>
  <dependency>
    <groupId>io.github.beehive-lab</groupId>
    <artifactId>tornado-runtime</artifactId>
    <version>4.0.0</version>
  </dependency>
</dependencies>
```

Tested in Cloud & Local Environments

```
ubuntu@ip-172-31-6-186:~/TornadoVM$ tornado --devices
WARNING: Using incubator modules: jdk.incubator.vector

Number of Tornado drivers: 1
Driver: OpenCL
  Total number of OpenCL devices : 1
  Tornado device=0:0 (DEFAULT)
    OPENCL -- [ComputeAorta] -- ComputeAorta AArch64
      Global Memory Size: 3.8 GB
      Local Memory Size: 32.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [1024]
      Max WorkGroup Configuration: [1024, 1024, 1024]
      Device OpenCL C version: OpenCL C 1.2 Clang 18.1.8
```



Graviton3 m7g

```
((.venv) ubuntu@ip-172-31-0-128:~/TornadoVM$ tornado --devices
WARNING: Using incubator modules: jdk.incubator.vector

Number of Tornado drivers: 1
Driver: PTX
  Total number of PTX devices : 1
  Tornado device=0:0 (DEFAULT)
    PTX -- PTX -- NVIDIA T4G
      Global Memory Size: 14.6 GB
      Local Memory Size: 48.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [2147483647, 65535, 65535]
      Max WorkGroup Configuration: [1024, 1024, 64]
      Device OpenCL C version: N/A
```



Graviton2 g5g

```
> tornado --devices
WARNING: Using incubator modules: jdk.incubator.vector

Number of Tornado drivers: 1
Driver: Metal
  Total number of Metal devices : 1
  Tornado device=0:0 (DEFAULT)
    METAL -- [Apple Metal] -- Apple M4 Pro
      Global Memory Size: 37.4 GB
      Local Memory Size: 32.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [1024]
      Max WorkGroup Configuration: [1024, 1024, 1024]
      Device OpenCL C version: N/A
```



Apple M4

```
> tornado --devices
WARNING: Using incubator modules: jdk.incubator.vector

Number of Tornado drivers: 2
Driver: SPIR-V
  Total number of SPIR-V devices : 2
  Tornado device=0:0 (DEFAULT)
    SPIR-V -- SPIRV OCL - Intel(R) Iris(R) Xe Graphics
      Global Memory Size: 28.7 GB
      Local Memory Size: 64.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [512]
      Max WorkGroup Configuration: [512, 512, 512]
      Device OpenCL C version: OpenCL C 1.2

  Tornado device=0:1
    SPIR-V -- SPIRV LevelZero - Intel(R) Iris(R) Xe Graphics
      Global Memory Size: 28.7 GB
      Local Memory Size: 64.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [512]
      Max WorkGroup Configuration: [512, 512, 512]
      Device OpenCL C version: (LEVEL ZERO) 1.3

Driver: OpenCL
  Total number of OpenCL devices : 2
  Tornado device=1:0
    OPENCL -- [NVIDIA CUDA] -- NVIDIA RTX A2000 8GB Laptop GPU
      Global Memory Size: 7.8 GB
      Local Memory Size: 48.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [1024]
      Max WorkGroup Configuration: [1024, 1024, 64]
      Device OpenCL C version: OpenCL C 1.2

  Tornado device=1:1
    OPENCL -- [Intel(R) OpenCL Graphics] -- Intel(R) Iris(R) Xe Graphics
      Global Memory Size: 28.7 GB
      Local Memory Size: 64.0 KB
      Workgroup Dimensions: 3
      Total Number of Block Threads: [512]
      Max WorkGroup Configuration: [512, 512, 512]
      Device OpenCL C version: OpenCL C 1.2
```

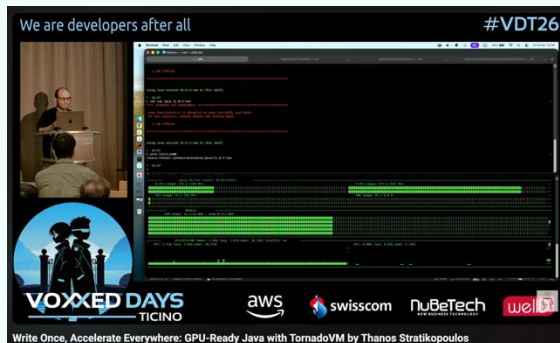


Precision 5570

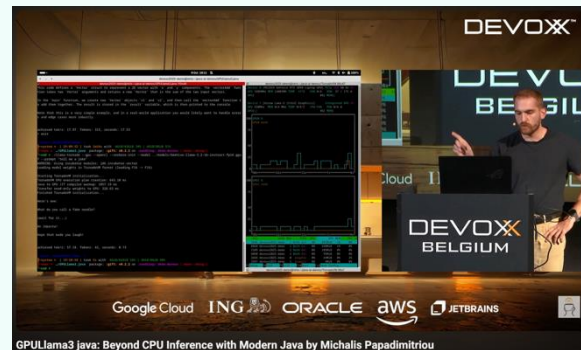
More about the technology



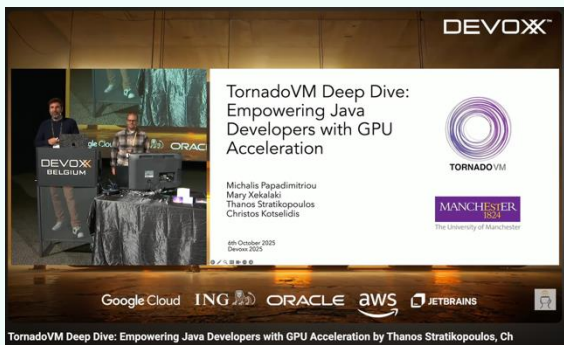
**Voxxed Days CERN
(2026)**



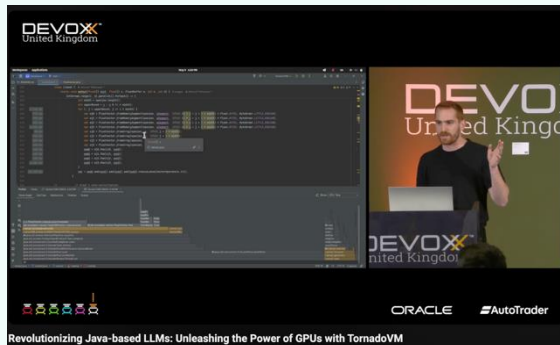
**Voxxed Days Ticino
(2026)**



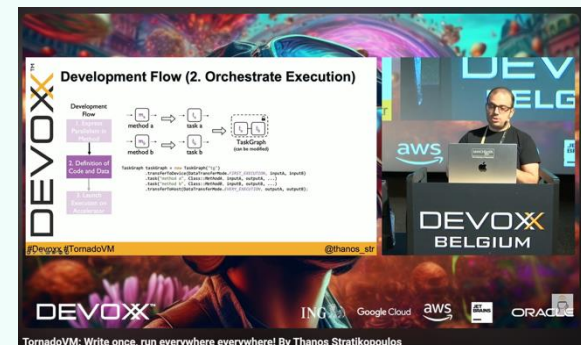
**DevOxx Belgium
(2025)**



**DevOxx Belgium
(2025)**



**DevOxx United Kingdom
(2024)**



**DevOxx Belgium
(2023)**

Tooling Ecosystem

Developer tools for building, profiling, and optimizing TornadoVM applications



TornadoInsight

IntelliJ IDEA plugin
Static analysis & testing



TornadoPulse

Profiling log analysis
Performance insights



TornadoViz

Bytecode visualization
Execution flow analysis



NVIDIA Nsight

GPU kernel profiling
CPU-GPU timeline



Intel VTune

Hotspot analysis
OpenCL/SPIR-V metrics



Docker

NVIDIA & Intel images
One-command install

All in-house tools are [open source: github.com/beehive-lab](https://github.com/beehive-lab)

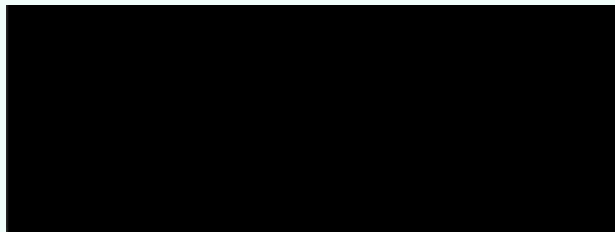
Full list: <https://tornadovm.org/tooling>



4

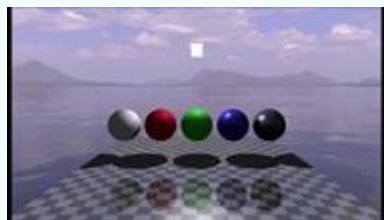
Use Cases & EU Research Projects

Use Cases



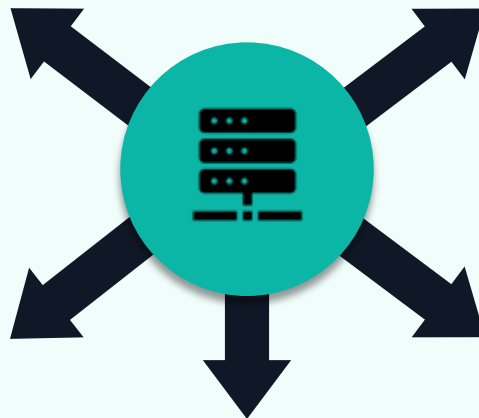
COMPUTER VISION

47x Performance



RAY TRACING

200x Performance



FACE DETECTION

23x Performance



MACHINE LEARNING

14x Performance



DATA ANALYTICS & NLP

Levenshtein Distance	9.8x
K-means Clustering	9.7x
Hierarchical Clustering	28x

GPULlama3.java

GPU-accelerated LLM inference in pure Java using TornadoVM

LangChain4j

AI framework integration

Quarkus

Cloud-native microservices

GGUF Models

Llama3, Mistral, Devstral 2,
Phi-3, Qwen, IBM Granite, Gemma (soon)



EU-Funded Research Projects

TornadoVM has been extended and adopted across multiple EU & UKRI funded research projects



European cloud infrastructure
ESA GAIA space data



P2CODE

Parallel programming
Developer tooling



Data sharing spaces
Energy-aware execution



Secure computing
Privacy-preserving ML



ELEGANT

Edge computing
Smart scheduling



E2Data

Big data processing
Heterogeneous infrastructure

Funded by [EU Horizon 2020](#) & [Horizon Europe](#), [UKRI](#), [Intel Corporation](#)
EPSRC grants: [PAMELA EP/K008730/1](#), [AnyScale Apps EP/L000725/1](#)



5

Sustainable Open-Source Practices

Building production-grade open-source technology within an academic organisation

The Six Pillars of Sustainable Open-Source

Practices that enable academic teams to ship production-quality software



GitHub PRs

Structured code review
for every contribution



Code of Conduct

Inclusive norms that
build community trust



Discussions

Slack + GitHub for
transparent decisions



GitHub Actions

Per-backend suites
across all JDKs
(per PR, Nightly, Weekly)



Contributor License Agreement (CLA)

Protection against legal disputes,
Project sustainability



Release Cycles

6+ releases with
automated distribution

Each pillar reinforces the others — together they form a self-sustaining development process

GitHub Pull Requests

The quality gate for every line of code entering TornadoVM

PR Workflow

Every contribution follows this flow

1. Fork or branch from develop
2. Implement + add regression tests
3. Open PR with clear description
4. Automated CI runs on all backends
5. Core team code review (1-2 reviewers)
6. Address feedback, iterate
7. Squash merge after approval

Key rules

No direct pushes to main/develop
All PRs require passing CI checks
External contributors welcome

Impact

10,000+

commits reviewed

30+

contributors worldwide

100%

PRs pass CI before merge

PRs are the single most impactful practice — they enforce quality, enable knowledge sharing, and create an audit trail

Code of Conduct

Setting the foundation for inclusive, productive collaboration

Why It Matters in Academia

Academic teams are unique

- PhD students rotate every 3-4 years
- Diverse cultural backgrounds
- Mixed experience levels (BSc to Prof)
- Remote + in-person collaboration

Without clear norms

- Miscommunication wastes research time
- Contributor churn increases
- Community growth stalls

Our Approach

Contributor Covenant adopted

- Respectful, harassment-free space
- Clear escalation paths
- Maintainer accountability

Practical outcomes

- Consistent PR review tone
- Constructive feedback culture
- Onboarding documentation for all
- Safe space for questions in Discussions

A Code of Conduct is not bureaucracy — it is infrastructure for trust across time zones and experience levels

Communication Channels

Combining real-time and asynchronous discussion for global collaboration

Slack

Real-time coordination

Quick questions, unblocking
Team standups and sync
Informal knowledge sharing
Channel: #tornadovm-dev



tornadovmcommunity.slack.com

GitHub Discussions

Persistent async Q&A

Design proposals and RFCs
User questions and support
Searchable knowledge base
Community-driven answers



github.com/bee-hive-lab/TornadoVM/discussions

GitHub Issues

Structured tracking

Bug reports with templates
Feature requests triaged
Linked to PRs and milestones
Labelled by backend/priority



github.com/bee-hive-lab/TornadoVM/issues

Slack for speed, Discussions for permanence, Issues for accountability — each channel has a clear purpose

Regression Testing

Catching breakages across backends, JDKs, and hardware before they reach users

Multi-Dimensional Test Matrix

Backend coverage

OpenCL – Intel, NVIDIA, AMD GPUs
PTX – NVIDIA CUDA
SPIR-V – Level-Zero dispatch
SPIR-V – OpenCL dispatch
Metal – Metal Shading Language

JDK distributions (21, 25)

OpenJDK, GraalVM,
Azul Zulu, Amazon Corretto,
Eclipse Temurin,
SapMachine, Microsoft OpenJDK,
Red Hat Mandrel

Hardware targets

NVIDIA, AMD, Intel GPUs
NVIDIA GPUs
Intel GPUs
Intel GPUs
Apple Silicon GPUs

4

backends tested
per commit

6+

JDK distributions
in nightly CI

0

untested PRs
merged to develop

GitHub Actions & CI/CD Workflows

Automating quality gates from commit to release

CI/CD Pipeline

```
// On every Pull Request (All backends, one JDK)
```

```
trigger: pull_request -> develop
  step: checkout + build all backends
  step: run OpenCL test suite
  step: run PTX/CUDA test suite
  step: run SPIR-V test suite
  step: checkstyle + static analysis
```

```
// Nightly (scheduled, All backends, all JDKs)
```

```
trigger: cron 01:00 UTC
  matrix: [OpenJDK, GraalVM, Temurin,
           Zulu, SapMachine, Corretto]
  step: full regression on all JDKs
```

```
// Weekly (Sundays, All backends, all JDKs, all devices)
```

```
trigger: cron 19:00 UTC
```

What This Enables

```
// Confidence to ship
```

Every PR verified against real GPUs
JDK compatibility guaranteed
Performance regressions caught early

```
// Reduced maintainer burden
```

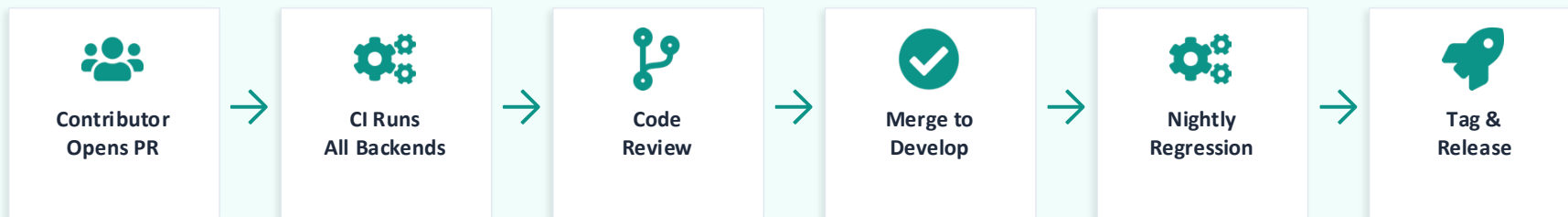
Automated checks = fewer manual reviews
Contributors see failures immediately
Self-service debugging via CI logs

```
// Reproducibility
```

Docker images for consistent envs
Pinned dependencies per release
Build matrix documented in YAML

How It All Connects

A virtuous cycle from contribution to release



Code of Conduct

Sets the tone for every interaction in the cycle

GitHub Discussions

Design decisions are discussed before PRs are opened

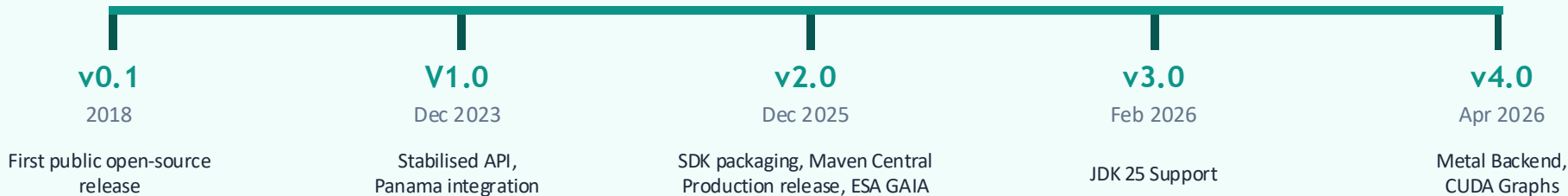
Slack

Unblocks contributors in real-time during review

The cycle is self-reinforcing: better processes attract more contributors, which improves the codebase further

Faster Release Cycles

From annual academic releases to continuous delivery



What Enables Fast Releases

Automation removes manual gates

CI validates all backends + JDKs
GitHub Actions builds + publishes
Docker images auto-generated

Distribution is instant

SDKMAN! sdk install tornadovm
Maven Central for library consumers
Docker images tagged per version

Quality never drops

No release without full regression pass
Nightly CI catches issues before tagging
Changelogs auto-generated from PRs

Results: Sustainable Open-Source in Academia

What these practices have delivered for TornadoVM

30+

releases since
2018 open-source

30+

contributors
worldwide

10,000+

reviewed
commits

1,400+

GitHub
stars

7+

EU & UKRI
research
projects

v4.0

latest stable
release

Key Message

Open-source best practices are not just for industry.

Academic projects can - and should:

adopt PRs, CI/CD, Codes of Conduct, CLA and structured communication from day one.

These practices are the reason TornadoVM went from a PhD project to a production framework
deployed in the ESA GAIA space mission.

The Elephant in the Room



It's Fun

Rewarding and impactful



But Grant-Funded

€0 when the grant ends



Dead Repos

Unmaintained code, lost community

Open-source academic software needs **sustainable funding models** beyond grant cycles,
or great code becomes a graveyard.

The Elephant in the Room



It's Fun

Rewarding and impactful



But Grant-Funded

€0 when the grant ends



Dead Repos

Unmaintained code, lost community

Open-source academic software needs **sustainable funding models** beyond grant cycles, or great code becomes a graveyard.

Initiatives That Show the Way - We Need More of These



Since 2010

EPSRC-funded • Phase 4 • £10.2M (2024–2028)
Edinburgh, Manchester, Southampton



EVERSE

2024-2027

Horizon Europe • Virtual Institute for SW
Excellence
18 partners • 10 countries • EOSC Science Clusters

NL NL eScience Center + EuSSI

Since 2012

NWO/SURF-funded • 60+ RSEs • €5.4M
sustainability fund
EuSSI: pan-European network for SW sustainability



Key Takeaways

- ✓ From a PhD project to a production-ready framework in a decade of innovation
- ✓ Multi-backend GPU acceleration: OpenCL, PTX, SPIR-V, Metal — without leaving Java
- ✓ Making Java AI-ready: GPULLama3.java, LangChain4j, Quarkus integration
- ✓ Rich tooling ecosystem and open-source community: 30+ contributors, Apache 2.0
- ✓ Real-world impact: ESA GAIA mission, 7+ EU/UK research projects (AERO, TANGO, P2CODE, ENCRYPT)
- ✓ Actively monitoring OpenJDK Babylon, Valhalla, Leyden for future platform integration



Questions?

